Paper Summary

A1. Paper Summary

Problem

Recent pre-trained models for time series, including TS foundation models and LLM-based adaptations, have shown strong zero/few-shot forecasting performance. However, many of these models can be computationally expensive (200-300M+ parameter), leading to slower inference times and dependency on expensive GPU hardwares. In addition, many do not explicitly support cross-channel correlations and exogenous signals that are essential for multivariate forecasting.

Core Idea and Methodology

In order to resolve this limitation, the paper proposes Tiny Time Mixers (TTM), a compact pre-trained model that can achieve strong zero/few shot forecasting performance while being extremely efficient and lightweight. To achieve this, TTM is built on TSMixer architecture which uses MLP-mixer blocks with simple matrix operations instead of computationally expensive transformer self attention. TTM scales this architecture down to very small sizes (around 1m parameters) while still maintaining transferability across many datasets.

Architecture Components:

TTM consists of four main components:

- TTM backbone for core processing using mixer-style blocks
- TTM decoder for efficient fine tuning (10-20% of backbone size)
- Forecast head consisting of single linear layer for final projection
- Optional exogenous mixer for incorporating external variables

Key pre-training modifications:

TTM introduces three key modifications to TSMixer to support effective pre-training:

1. **Adaptive Patching (AP)**: TTM uses multi-level backbone where the representation is processed at multiple patch granularities. The backbone changes how patches are arranged across levels resulting in some levels operating on more but smaller patches and others operating on fewer but larger patches. Through structured partition and merge operations around TSMixer blocks, the model can view the same sequence at different temporal granularities with minimum computing cost. This design helps the

model generalize across datasets that prefer different patch sizes and sampling characteristics.

2. **Diverse Resolution Sampling (DRS)**: During pre-training, imbalance can occur when high-resolution datasets dominate the training corpus because they simply contain more timestep samples. This can lead to a bias towards finer resolution data. DRS avoids this by augmenting the training set with additional lower resolution series generated from high-resolution series. This is done using downsampling strategies such as 1) averaging k samples in sequential, non-overlapping windows to produce a lower resolution dataset; and 2) conventional decimation where only every kth sample is retained. This increases coverage across multiple resolutions and reduces bias toward fine-grained datasets, while still keeping the original high-resolution data in the pre-training pool.

3. **Resolution Prefix Tuning (RPT)**: RPT provides explicit resolution awareness by embedding each temporal resolution as a learnable prefix concatenated with the first input patch. This enables resolution-conditioned modeling where the model can adapt its processing based on the resolution. This approach is particularly useful when training across diverse resolutions with limited model capacity and when short context windows make resolution difficult to infer from the sequence alone.

Fine Tuning:

During fine-tuning, a target-domain dataset is used which does not overlap with the pre-training dataset. The backbone remains frozen during fine-tuning and continues to operate in a channel independent manner. Adaptation is primarily handled by the TTM head which contains the decoder and forecast head. Fine-tuning optimizes the same forecasting objective using MSE-loss and the forecast head produces the final predictions.

Depending on the target tasks, the decoder can be configured differently. For univariate target tasks, the decoder can operate the channel independently while for multivariate target tasks, channel mixing can be enabled inside the decoder's mixer blocks to explicitly model cross-channel correlations.

Furthermore, if the target dataset contains exogenous variables whose future values are known in advance, TTM applies an Exogenous Mixer Block after generating the initial forecasts to refine predictions. The process works as follows:

- The model produces an initial multi-channel forecast.
- The predicted values of the exogenous features are replaced with their true future values.
- The resulting sequence is re-patched using a series of overlapped windows (creates a new tensor) to learn inter-channel lagged correlations.
- The tensor is passed through a vanilla TSMixer block with channel mixing enabled and then applied to a final linear head to output refined forecasts for the target variables.

This mechanism allows the model to learn how exogenous signals influence the targets, including lagged effects, without changing the frozen backbone.

Main Contribution and Claims

TTM is evaluated on standard forecasting benchmarks (D1), demonstrating that TTM variants achieve competitive or superior zero-shot accuracy and improve further under few-shot fine-tuning using only 5-10% of target data, validating the transfer learning capability. Cross-transfer experiments (ETTh2→ETTh1) show that TTM's pre-training yields strong transfer performance compared to several pre-training baselines, confirming that performance gains stem from generalizable pre-trained representations.

A second major claim is practical efficiency: the paper reports comparisons of parameter count (1M vs. 200-300M), fine-tuning cost (65X faster), and inference time (0.01s vs. 0.4-2300s), demonstrating that TTM is substantially more efficient than competing models while maintaining superior accuracy. The model is also lightweight enough to run on CPU-only hardware without requiring GPUs.

Additionally, the paper claims that TTM supports true multivariate forecasting by incorporating cross-channel correlations and exogenous signals during fine-tuning, validated through evaluation on benchmark set D2 where enabling channel mixing and exogenous fusion improves performance.

These claims are further supported by ablation studies showing that

- Pre-training significantly outperforms random initialization, especially in low-data settings (21% improvement with 5% data and 5 epochs).
- Adaptive Patching improves robustness across diverse datasets by enabling multi-granularity learning.
- Diverse Resolution Sampling provides substantial performance gains (37% improvement) by augmenting resolution coverage.
- Resolution Prefix Tuning is most beneficial when context windows are short and resolution cannot be easily inferred from the input sequence.

Strengths

- **Overcomes accuracy-efficiency trade-off:** The experiments show that TTMs achieve better zero/few-shot forecasting accuracy compared to popular SOTA forecasting models such as TimesFM, Moirai, Chronos, and Lag-Llama while staying very compact and practical to run.

- **Clear evidence on standard benchmarks:** The evaluation is done on a widely used set of seven public datasets (D1: ETTh1/ETTh2/ETTm1/ETTm2/Weather/Electricity/Traffic) under zero-shot and few-shot settings, which makes results easier to compare with prior work.

- **Clear evidence on efficiency claim:** Explicit comparisons of Inference time per-batch in GPU and CPU, total parameters (Params), and maximum GPU memory usage (MEM), highlighting that TTM is among the fastest and lowest-memory options in their comparisons.

- **Ablations justify the proposed modifications:** The ablation section isolates the effect of pre-training vs. random initialization (21-36% gains), AP (4% improvement), DRS (37% improvement), and RPT (3% improvement), connecting architectural choices to performance gains.

- **Explicit path for multivariate and exogenous modeling:** Pre-training is channel-independent, but fine-tuning enables channel mixing and exogenous signal integration through modular components that can be activated on demand. This design provides flexibility where simpler tasks can use the frozen backbone alone, while complex multivariate scenarios benefit from enabling the decoder's channel mixing and exogenous mixer. The paper validates this capability on benchmark set D2.

Weaknesses and Limitations

- **Zero-shot may underuse multivariate structure:** Since pre-training is channel-independent and explicit channel mixing and exogenous infusion are primarily introduced during fine-tuning, zero-shot forecasts may not fully exploit cross-channel dependencies or exogenous covariates in target-domain multivariate settings.

- **Missing regime shift stress tests:** Evaluation is limited to standard benchmarks with stable temporal patterns. TTM's behavior under regime shifts or structural breaks in real-world scenarios still remains to be tested. Additional experiments centered on structural breaks or known shock periods (covid-19 or market crash periods) would better quantify whether TTM's pre-trained representations remain reliable or degrade sharply under shift.

- **Limited Support for Irregular Time Series:** DRS and RPT assume data arrives at regular, well-defined temporal resolutions that can be mapped cleanly to discrete resolution IDs. However, real-world time series can exhibit irregular sampling, missing observations, or mixed frequencies that don't fit this framework, requiring manual preprocessing for ambiguous data such as business day schedules or sensor noises. Furthermore, DRS relies on averaging and decimation for downsampling, which may lose domain-specific patterns such as peak values or rare events.

- **Channel Independent Backbone can limit multivariate learning:** Although pre-training the backbone in a channel-independent manner and deferring all cross-channel modeling to the lightweight decoder during fine tuning is efficient, this design may underperform on datasets where cross-channel dependencies are essential throughout multiple representation layers since the backbone can never learn shared multivariate features that could transfer across domains.

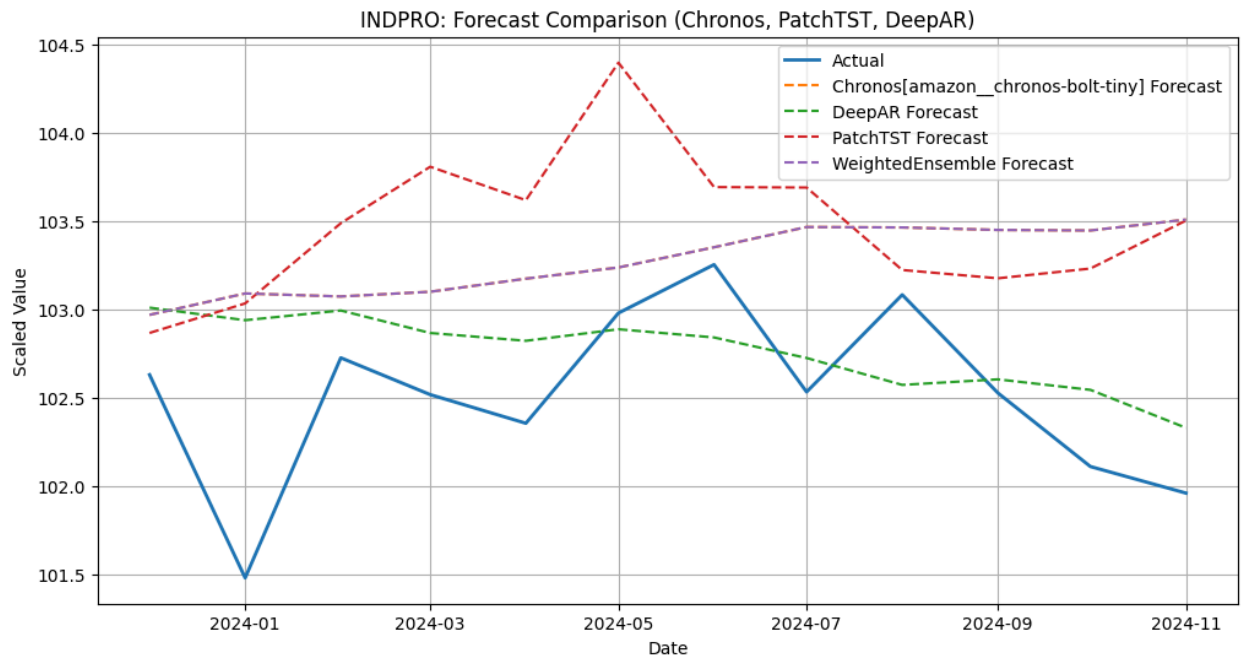
What's Next

- **Architecture Change that we tried out:** We changed the amount of hidden dimensions from the default of 64 - 128. By doing this and then pretraining the model we were able to see the best result across all models we tested. The intuition behind this is since we are trying to forecast all 126 time series that FRED gives us we could potentially benefit by having more dimensions. This seemed to be the case as we saw the best results with this setup.
- **Pretraining using the sliding window technique they used in the paper:** We could increase the context window and use the sliding window technique to better pretrain the model. This would lead to more training time and more setup time for the data. The same evaluation metrics can be used to see if this made any differences and we can plot the predictions to compare with the actual trend.

Experiment Write Up

Testing 3 TSFM Models

- FRED-MD_2024m12.csv for zero-shot testing
- Take 512 month context window (4-1-1981 to 11-1-2023) and try to predict next 12 months up till (11-1-2024)
- We also forecast all 126 time series at the same time
- We standard scale all the data so the model is not biased towards larger numbers
- The data was then converted into pytorch tensors
- We benchmark 3 models Chronos bolt tiny, PathTST, and Deep AR using the autogloun library.
- The evaluation metrics used include RMSE, MAE, MASE.
- Below is the result for the 3 models compared to the baseline. The weighted ensemble is just chronos in this case.

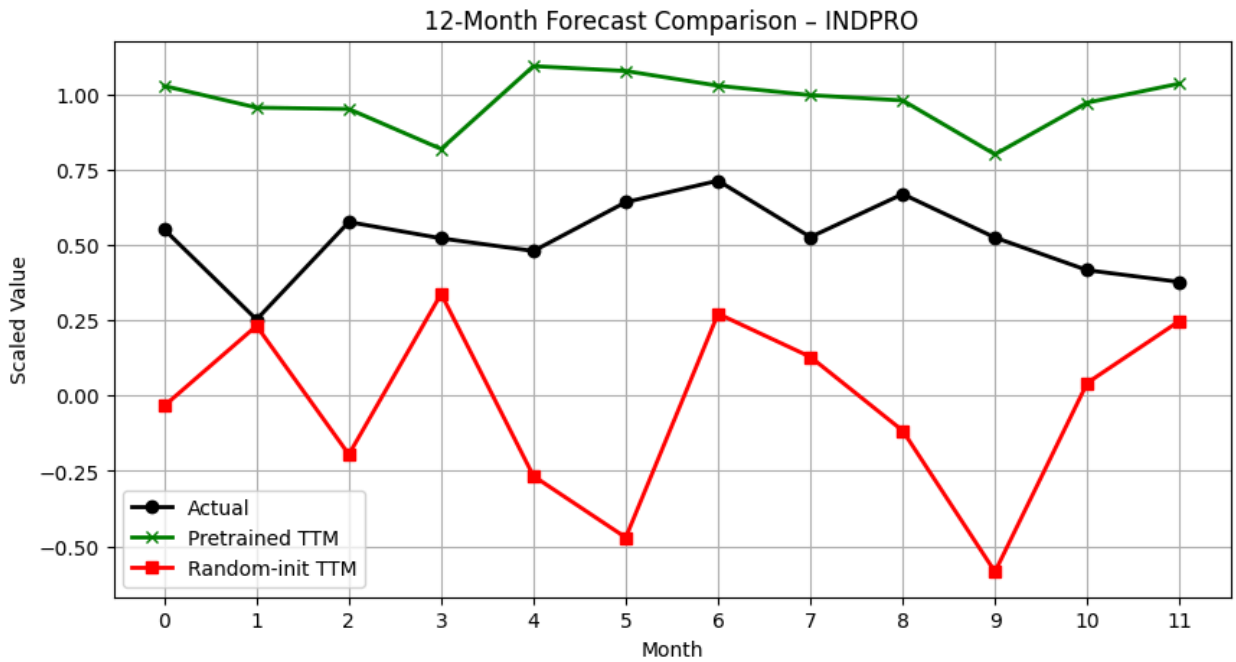


--- RMSE Comparison ---
Chronos[amazon__chronos-bolt-tiny] RMSE: 0.91
DeepAR RMSE: 0.54
PatchTST RMSE: 1.08
WeightedEnsemble RMSE: 0.91

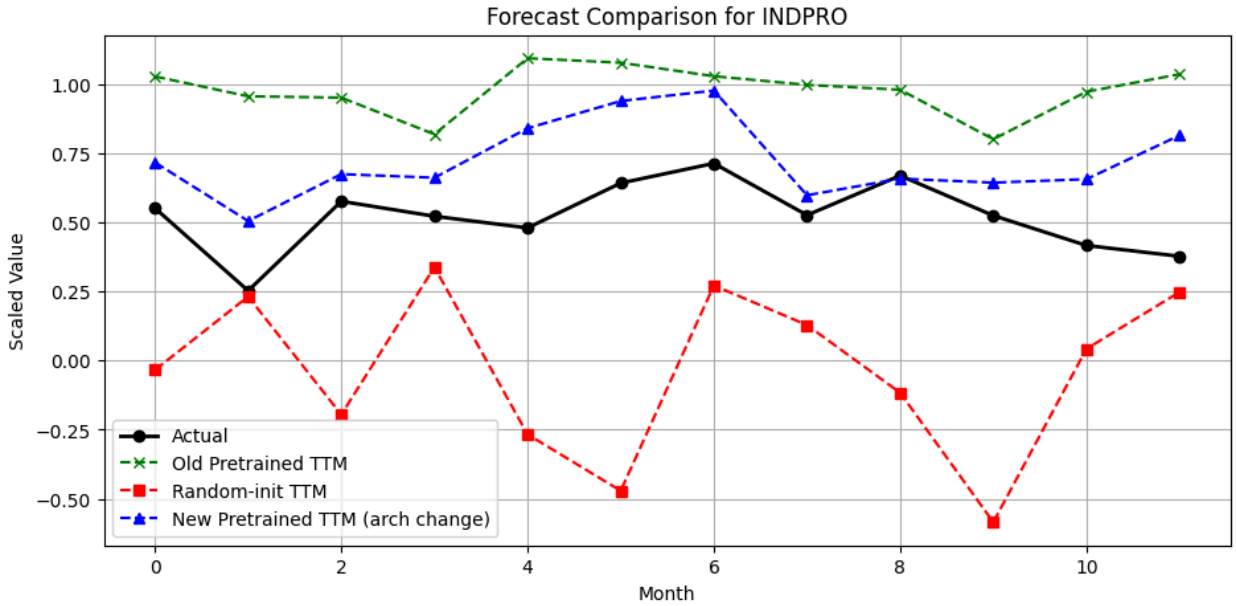
DeepAR had the lowest RMSE (would also be lowest MSE). PathTST had the worst performance.

Pretraining TTM, Testing generic TTM, architecture change + pretraining

- FRED-MD_2024m12.csv for zero-shot testing
- Take 60 month context window (12-1-2018 to 11-1-2023) and try to predict next 12 months up till (11-1-2024)
- We also forecast all 126 time series at the same time
- We standard scale all the data so the model is not biased towards larger numbers
- The data was then converted into pytorch tensors
- We configure the TinyTimeMixerConfig with input size 126, context_length = 60, prediction_length = 12.
- The model is trained multivariate forecasting with MSE for the loss and Adam for the optimizer for 50 epochs.
- The model is then evaluated obtaining a MSE of 0.555197.
- A non-pretrained TTM is evaluated and we get a MSE of 1.8264.
- Very clear indication that the pretraining led to some sort of improvement



- For the architecture change we made a TTM with increased hidden dimensions from the default of 64 to 128. We did the same training strategy on this as the other one.
- This pretrained model got a MSE of 0.314932
- The change in architecture made a pretty sizeable difference
- You can see the predictions it made do a much better job of following the actual and even replicates a of of the dips and up swings of the actual predictions.


Forecast Comparison for INDPRO
Scaled Value
Month
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
0
2
4
6
8
10
Actual
Old Pretrained TTM
Random-init TTM
New Pretrained TTM (arch change)